Material complementar

PROGRAMAÇÃO NO PROFITCHART 1.0



O PRIMEIRO CANAL DE PROGRAMAÇÃO AVANÇADA NO PROFITCHART APRESENTA

PROGRAMAÇÃO NO PROFITCHART 1.0

DOMINANDO O EDITOR DE ESTRATÉGIAS

Um curso detalhado desde o básico até o avançado sobre tudo que você precisa saber para construir estratégias e fazer diversos backtests utilizando a linguagem de programação da Nelogica !

QUAL O OBJETIVO DESTE MATERIAL COMPLEMENTAR?

Esse material faz parte do curso "Desenvolvendo Setups para Vencer na Bolsa de Valores" e tem como objetivo ajudar aqueles que estão iniciando na programação a entenderem os conceitos básicos do Editor de Estratégias.



Desenvolvendo Setups para Vencer na Bolsa de Valores

Módulo 1 – Começando com a programação em NTSL (3 aulas):

1° Aula – Áreas do código, declaração de variáveis palavras reservadas e operadores

Áreas do código

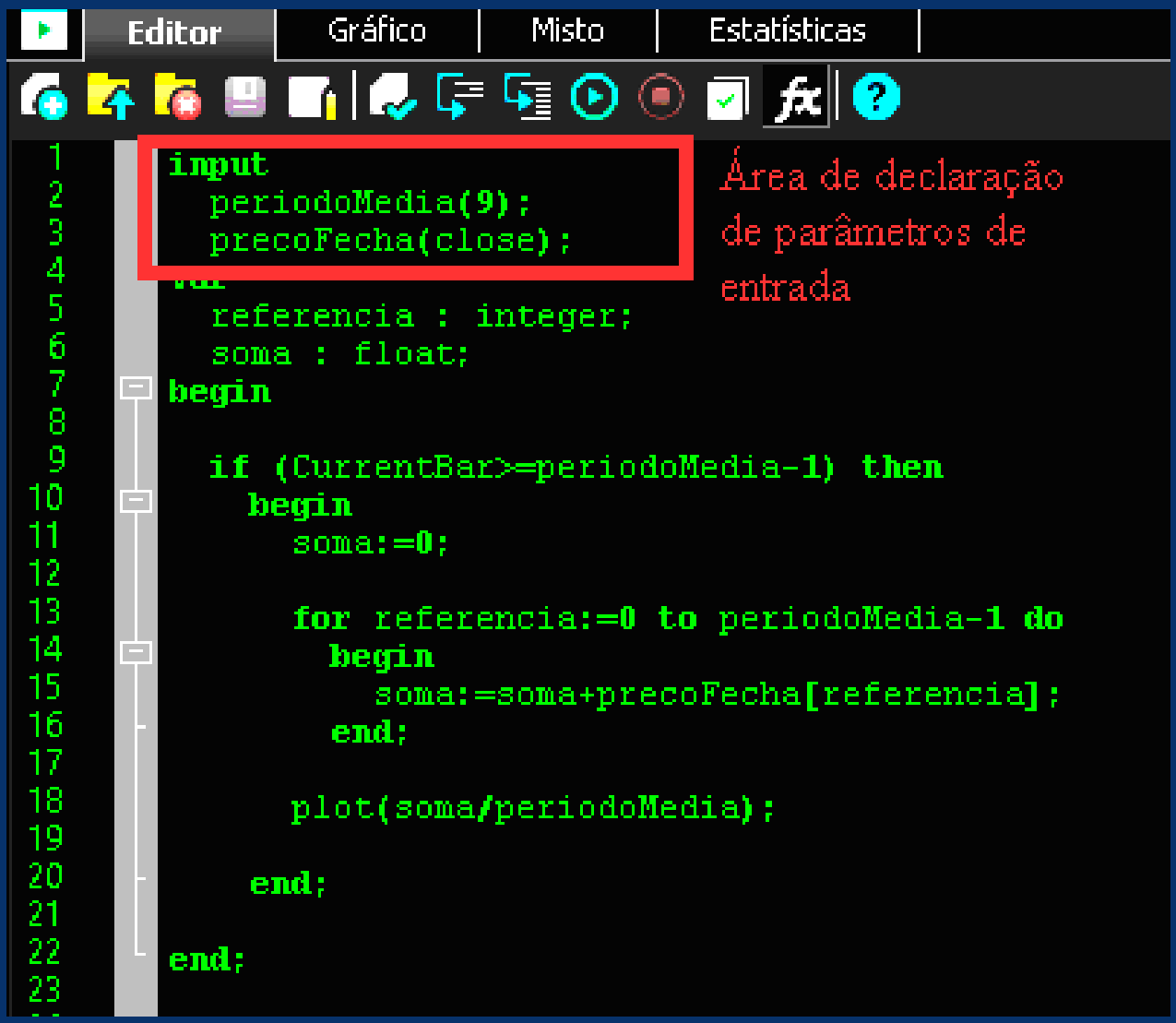
Existem três áreas fundamentais que definem a estrutura de uma estratégia, observe:

1. Área de declaração de parâmetros de entrada (input);
2. Área de declarações de variáveis (var);
3. Área do código (begin e end);

Para você que está acostumado a assistir aos vídeos de programação lá no canal, já deve ter percebido que eu sempre repito o mesmo processo quando início a escrita do código. Lembra das palavras input, var, begin e end?

Quando escrevemos a palavra input estamos abrindo um bloco para informar todos os parâmetros externos que a estratégia usará. Esses parâmetros são muito úteis porque, por meio desta única parte no código, podemos alterar várias coisas rapidamente, por exemplo, alterar o período da média em um código com 200 linhas onde essa média é mencionada em diversos locais distintos no corpo do código. Se não usássemos o input, isso deveria ser feito vagarosamente em cada parte do código em que média estivesse sendo usada.

Para usar o input, devemos apenas atribuir um nome ao parâmetro e abrir os parênteses com o ponto e vírgula, dentro destes colocaremos o valor que queremos atribuir ao parâmetro criado. Feito isso, todos os locais do código que contenham o nome criado serão equivalentes aquilo que foi colocado dentro dos parênteses.




```
input
  periodoMedia(9);
  precoFecha(close);
var
  referencia : integer;
  soma : float;
begin

  if (CurrentBar>=periodoMedia-1) then
    begin
      soma:=0;

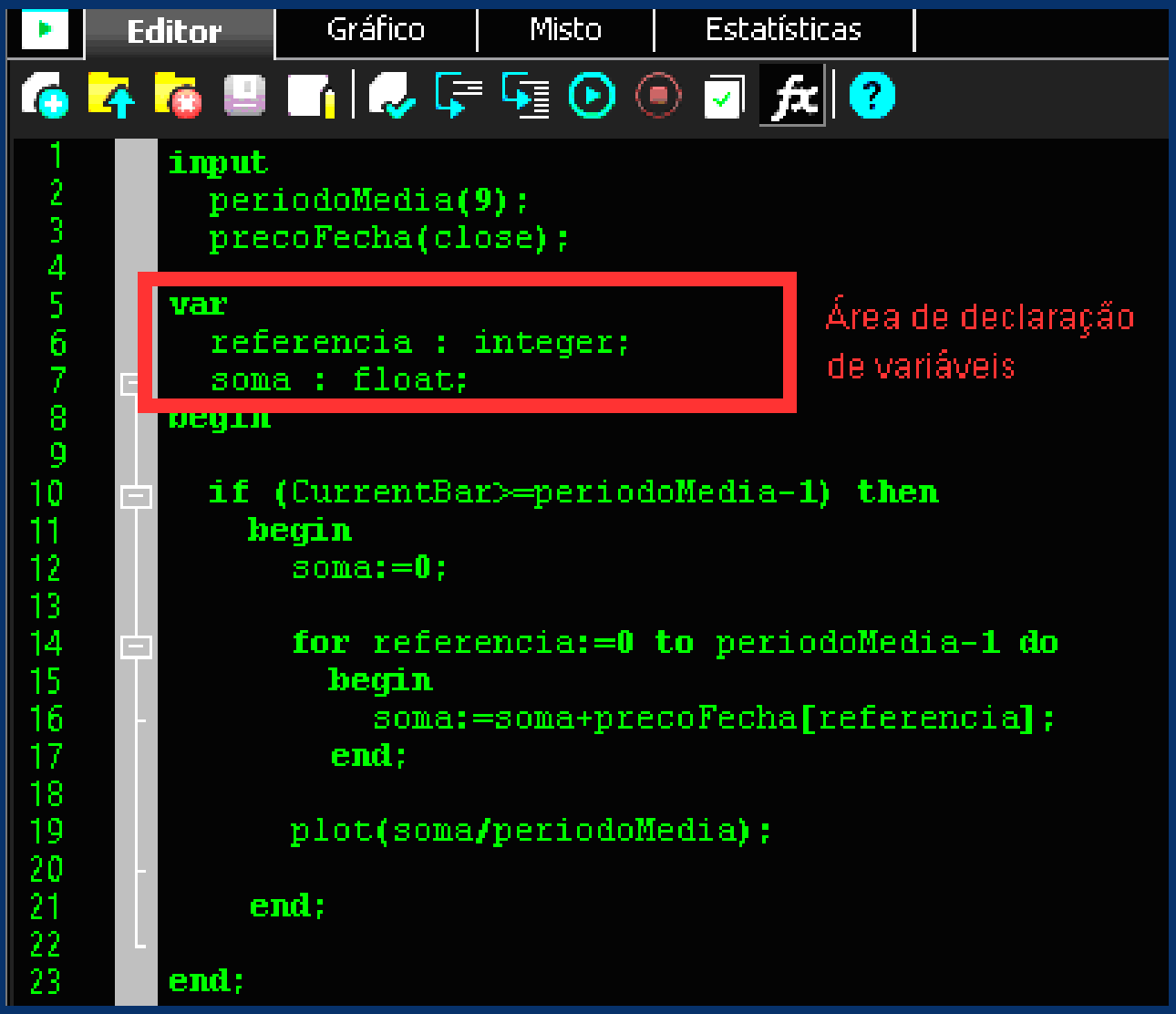
      for referencia:=0 to periodoMedia-1 do
        begin
          soma:=soma+precoFecha[referencia];
        end;

      plot(soma/periodoMedia);

    end;

end;
```

A segunda área que precisamos aprender é a área de declaração das variáveis. Ela é atribuída pela instrução VAR seguida da relação de variáveis. Após os nomes de cada variável deverá ser utilizado o símbolo dois-pontos (:), e após estes é mencionado o tipo de dado que a variável irá receber, seguido de ponto-e-vírgula. Observe:

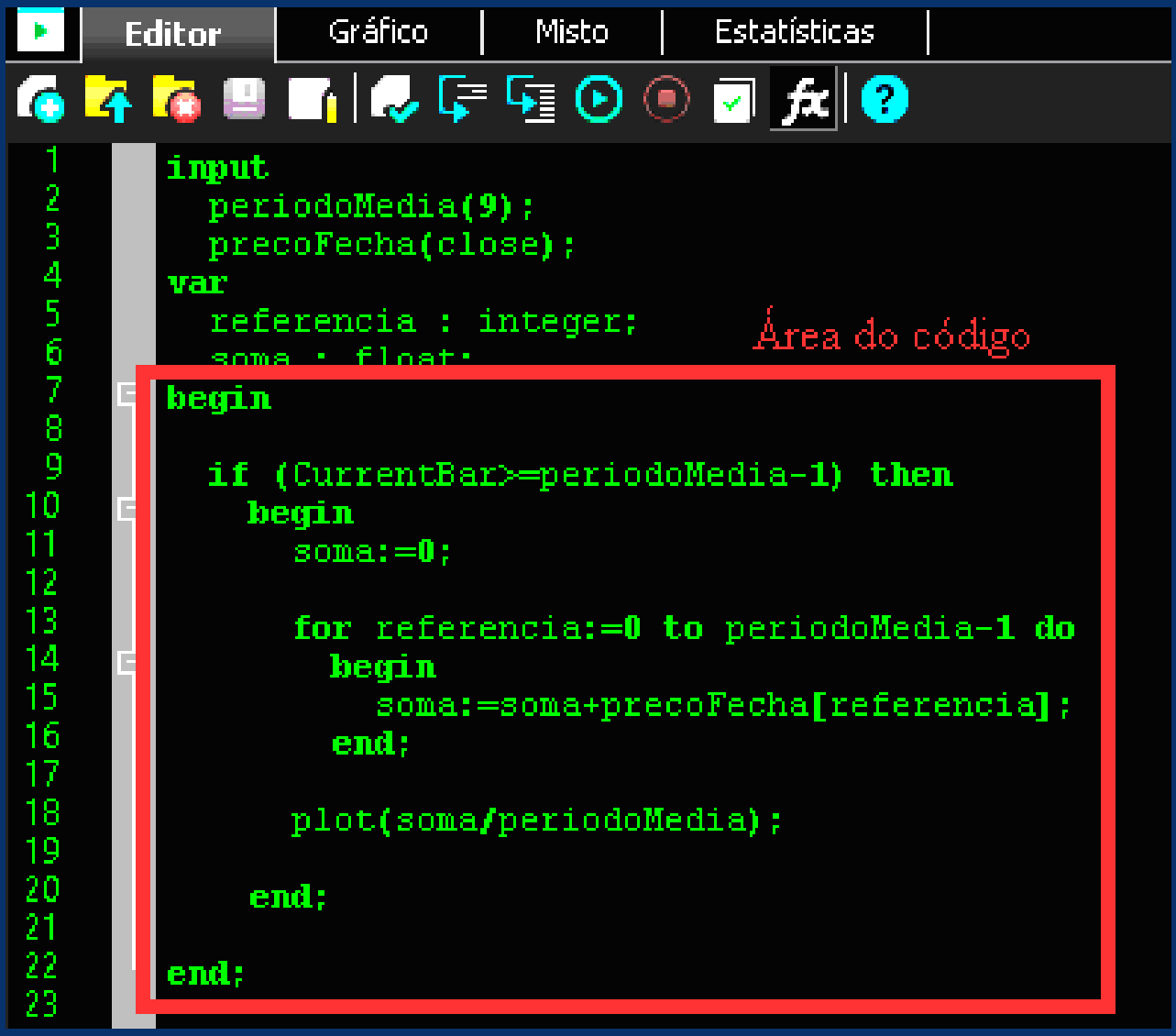




Caso as variáveis sejam do mesmo tipo, estas poderão ser relacionadas separadas por vírgula.



Por último, temos a área do código, também chamada de corpo do programa. É nesse local que vamos programar a estratégia e qualquer outra coisa que quisermos fazer no editor. Essa área é a principal e é onde passaremos a maior parte do tempo programando. Perceba que ela se inicia com a instrução begin e é finalizada com a instrução end seguida do ponto e virgula. Observe:



Declaração de variáveis

A tarefa das variáveis é armazenar valores que serão utilizados na estratégia, para isso nós precisamos definir qual é o tipo de variável que vamos precisar. Existem três tipos:

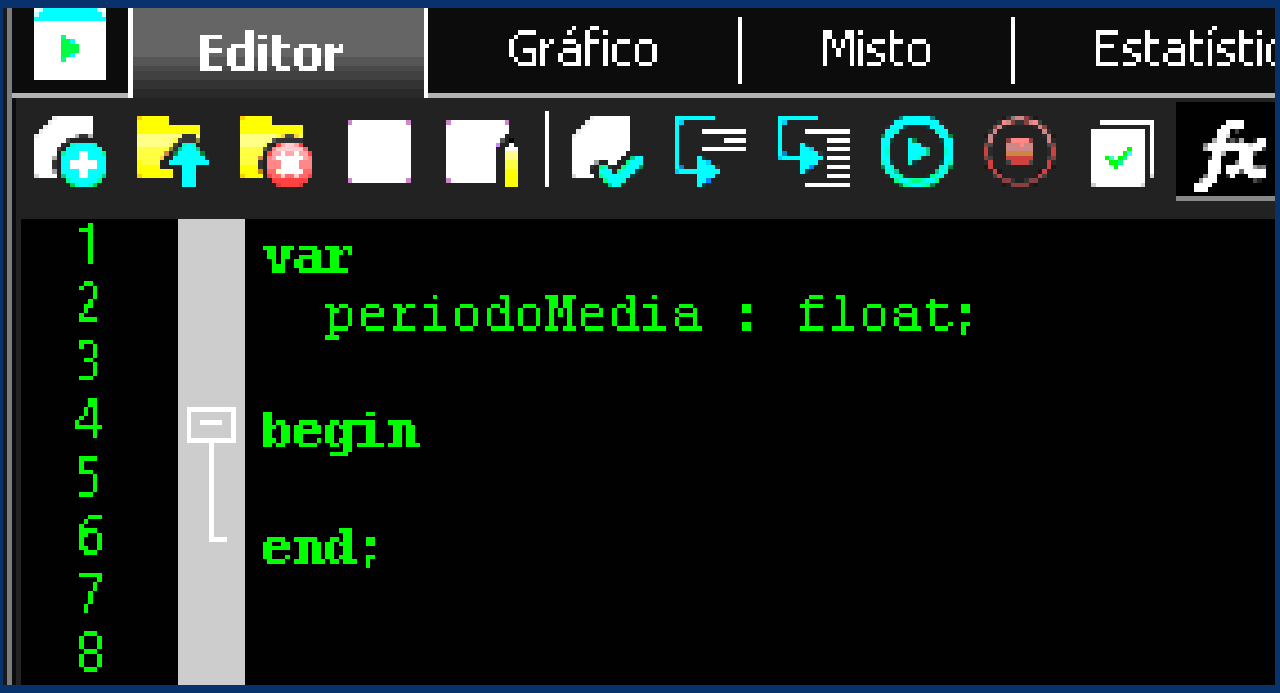
- Integer – Representa números inteiros, ou seja, números que não possuem vírgula. Por exemplo, número de dias, quantidade de operações, período de um indicador, etc.

- Float – Representa números com vírgula, por exemplo, valor que um indicador fornece, preço de abertura, preço de fechamento, etc.

- Boolean – Representa um tipo booleano. Resumindo, pode retornar verdadeiro ou falso.

É muito importante entender que as variáveis na linguagem NTSL oferecem uma flexibilidade muito útil. Dessa forma, podemos acessar valores atuais e passados facilmente. Esse conceito é de extrema importância, por meio dele podemos acessar o valor de uma média de 5, 10, 30 ou muitos dias atrás!

Vamos declarar uma variável chamada periodoMedia que será do tipo float. Fazemos isso na área nomeada por VAR, escrevemos o nome da variável à esquerda, colocamos dois pontos e o tipo da variável fica na direita. É simples, observe:



Para acessar valores passados dessa variável precisamos entender qual é a dinâmica, perceba que hoje seria equivalente ao número zero entre colchetes, ontem seria o número um entre colchetes e assim por diante. Observe:

```
var
  periodoMedia : float;

begin

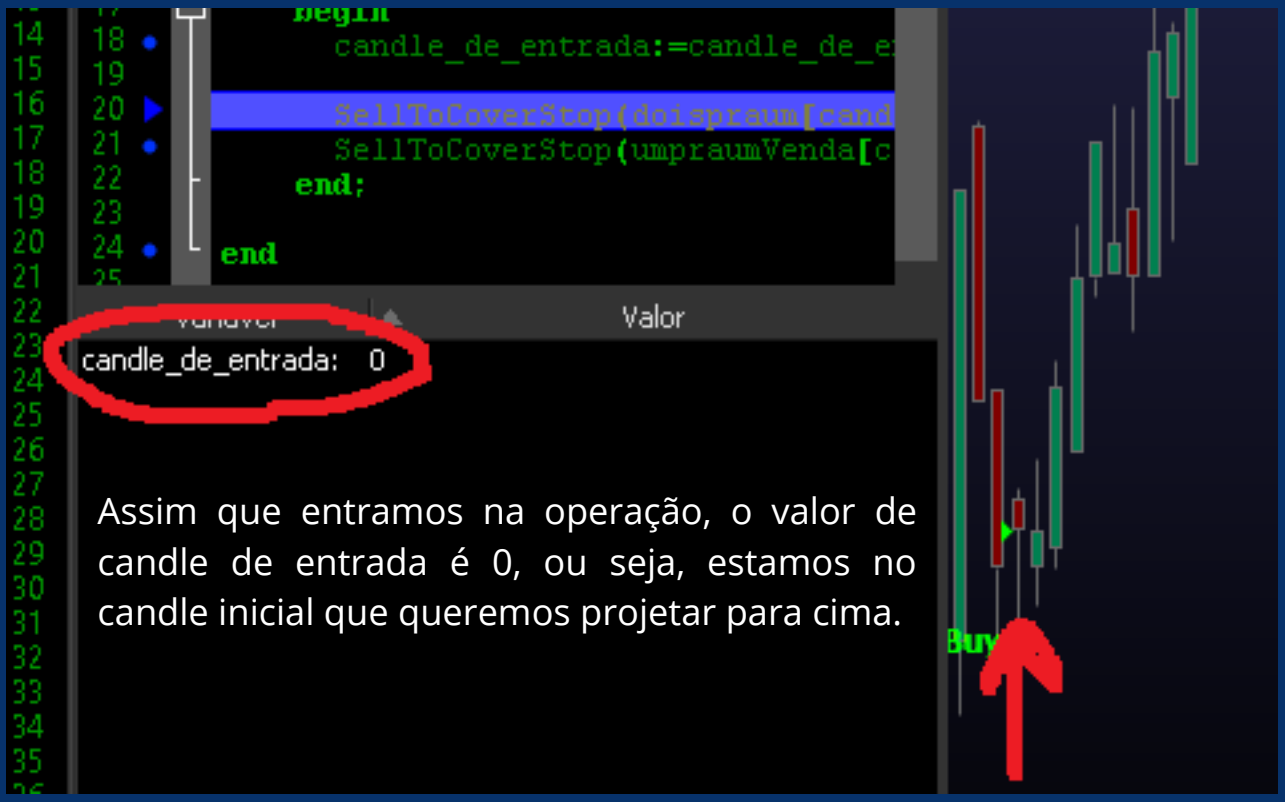
  periodoMedia:=MediaExp(9,close)[0]; // Preço da média hoje
  periodoMedia:=MediaExp(9,close)[1]; // Preço da média ontem
  periodoMedia:=MediaExp(9,close)[2]; // Preço da média anteontem
  periodoMedia:=MediaExp(9,close)[3]; // Preço da média 3 dias atrás

end;
```

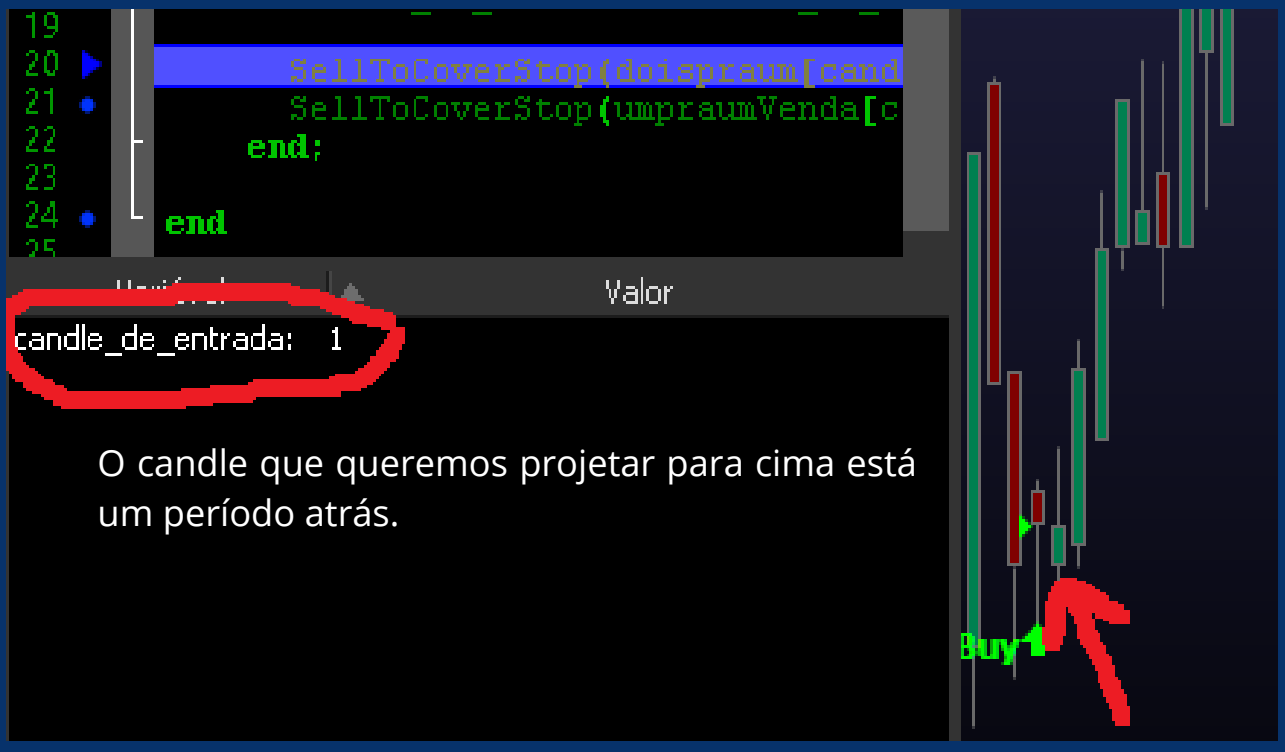


Esse conceito é utilizado constantemente quando queremos marcar o candle de entrada, isso porque conforme novas barras vão se formando, nosso candle de entrada vai se distanciando e senão tivermos um bom código, perderemos a referência de onde entramos na operação, o que seria um grande problema.

Basicamente, ao entrar em uma operação, precisamos iniciar uma contagem de candles para marcar nosso candle de entrada. Por meio dessa contagem, não importa quanto tempo estamos posicionados, ele sempre vai saber quantos candles surgiram após a entrada.



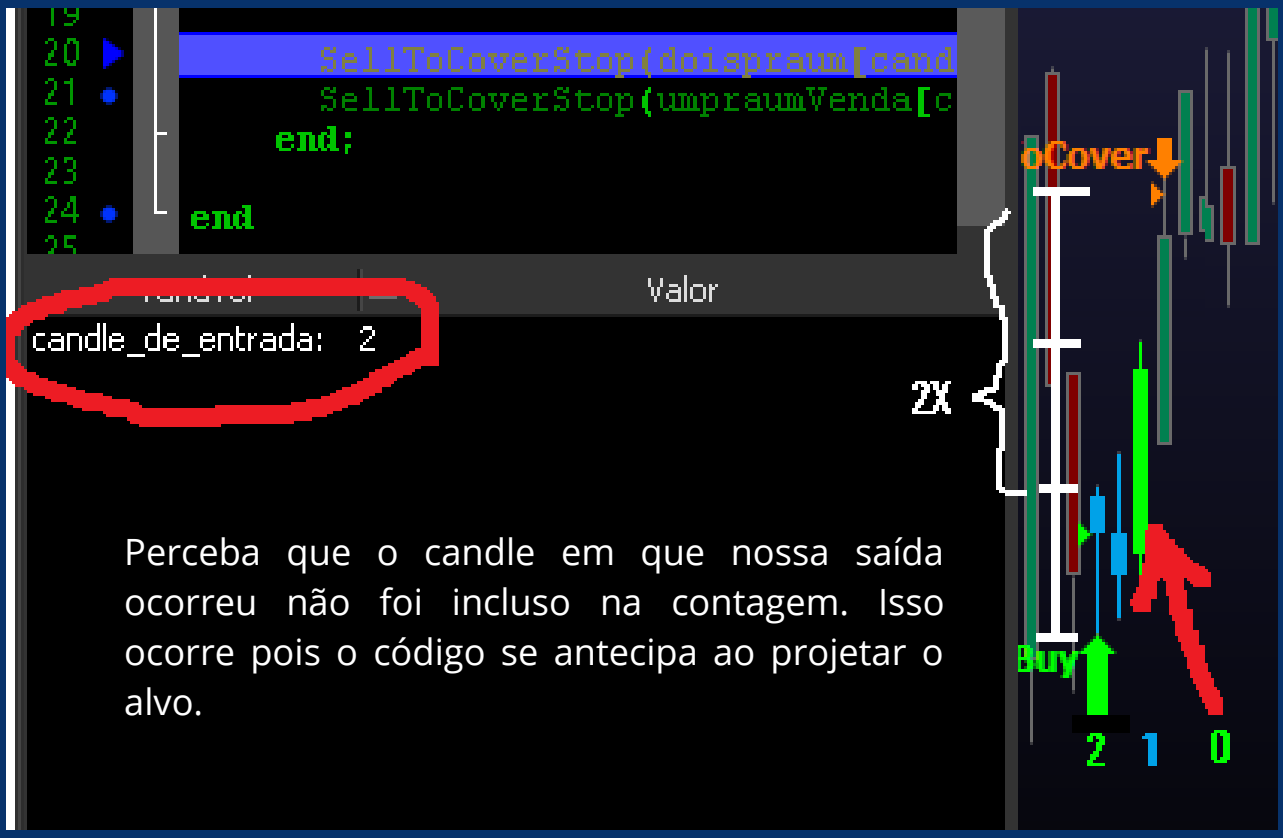
Assim que entramos na operação, o valor de candle de entrada é 0, ou seja, estamos no candle inicial que queremos projetar para cima.



O candle que queremos projetar para cima está um período atrás.

Observe como programamos o alvo. A função dois para um pode ser criada separadamente ou podemos escrever os cálculos do alvo ali mesmo.

```
SellToCoverStop(doispraum[candle_de_entrada]+100,doispraum[candle_de_entrada]);
```



Palavras reservadas

A linguagem NTSL reconhece certo grupo de palavras como sendo reservadas. Essas palavras tem significado especial e não podem ser usadas como identificadores em um programa, ou seja, nome de variáveis. A seguir estão todas as palavras reservadas do NTSL:

- and
- array
- begin
- const
- do
- downto
- else
- end
- for
- function
- if
- of
- or
- procedure
- repeat
- then
- to
- until
- var
- while
- input

Operadores aritméticos

São simples de entender, tratam-se da adição, subtração, multiplicação e divisão.

Operador	Descrição
+	Adição
-	Subtração
*	Multiplicação
/	Divisão

Operadores relacionais

Os operadores relacionais são utilizados na comparação entre dados de mesmo tipo, observe:

Operador	Símbolo
Maior que	>
Menor que	<
Maior ou igual	>=
Menor ou igual	<=
Igual	=
Diferente	<>

Operadores lógicos

Extremamente úteis para definir condições de entrada e saídas das estratégias, já que é muito comum mais de uma condição ser verdadeira para entrar ou sair de uma operação. Então, observe abaixo quais são esses dois operadores:

- AND – Significa “e”, por exemplo, se a média de nove estiver acima da média de vinte E o estocástico lento estiver maior que 50, deve-se entrar na operação.

- OR – Significa “ou”, por exemplo, se o preço de fechamento estiver maior que a média de nove OU o estocástico lento for maior que 50, deve-se entrar na operação.

- NOT – Utilzado para negar uma condição. Por exemplo, se o fechamento não estiver acima da média de 50, não entrar na operação.



2° Aula – Controlando o fluxo do programa

Existem algumas palavras reservadas essenciais para que possamos controlar o fluxo de execução do programa e assim criar estratégias no Editor de Estratégias do ProfitChart, são elas: IF, FOR e WHILE.

Estrutura condicional IF

Quando usamos o IF estamos criando uma maneira da estratégia realizar decisões básicas. O que nos iremos fazer é colocar o computador para fazer um teste, se o teste for verdadeiro ele executa certa parte do código, se o teste for falso ele não executa. Resumindo, dependendo do resultado do teste é que o computador irá decidir se executa determinada parte do código ou não. Existem dois tipos:

- Estrutura simples - A instrução IF simples tem por finalidade tomar uma decisão e efetuar um desvio no processamento, que ocorre de acordo com a resposta para o teste dentro dos parênteses. No caso da estrutura simples, utilizamos apenas o IF...THEN.

```
if ( teste ) then
  begin

    //Entra aqui caso o teste
    //seja verdadeiro

  end;
```

- Estrutura composta - A instrução IF composta tem por finalidade promover um desvio no processamento não só caso a resposta do teste resulte em verdadeiro, ela também permite definir uma instrução a ser executada caso a resposta para o teste seja falso. Nesse caso, apenas precisamos incluir a palavra reservada ELSE.

```
if ( a > b ) then
  begin

    // Instrução se o teste
    // retornar verdadeiro

  end

else
  begin

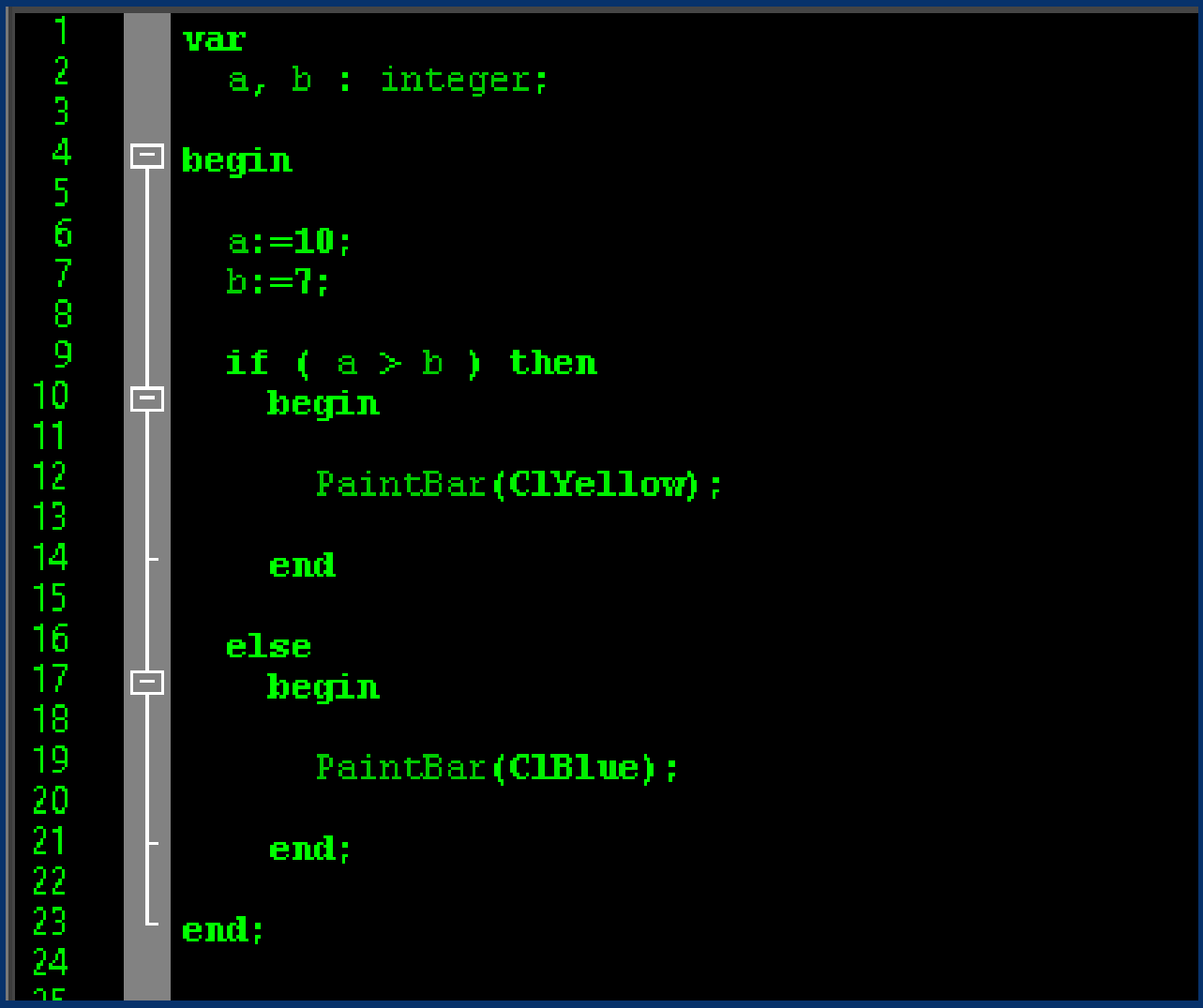
    // Instrução se o teste
    // retornar falso

  end;
```



Não esqueça de colocar o ponto e vírgula (;) no lugar correto. Quando incluímos o ELSE na estrutura, precisamos retirar o ponto e vírgula do END anterior ao ELSE e coloca-lo no final da estrutura condicional.

Para facilitar o entendimento, vejamos o exemplo abaixo onde temos dois valores a serem colocados dentro de uma estrutura condicional:



```
var
  a, b : integer;

begin

  a:=10;
  b:=7;

  if ( a > b ) then
    begin

      PaintBar(ClYellow);

    end

  else
    begin

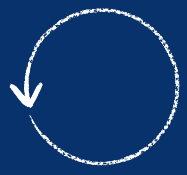
      PaintBar(ClBlue);

    end;

end;
```

No exemplo acima temos duas variáveis chamadas "a" e "b". Armazenamos o valor 10 na variável "a" e o valor 7 na variável "b". Se "a" for maior que "b", então ele pinta todas as barras de amarelo, senão, então todas as barras serão pintadas de azul. Dessa forma, concluímos que todas serão pintadas de amarelo, pois o número 10 é maior do que o número 7.

Laço de repetição WHILE



Quando queremos selecionar uma parte do código e fazer com que ela execute várias e várias vezes, podemos ir pelo caminho mais difícil e escrever dezenas de linhas de código ou podemos simplesmente jogar essa parte do código em uma estrutura chamada laço de repetição.

A estrutura WHILE realiza um teste lógico e enquanto o teste for verdadeiro ela permanece repetindo o que está dentro dela. Observe como escrevemos um laço de repetição WHILE...DO:

```
while ( a < b ) do
  begin

    // Instruções

  end;
```

No exemplo abaixo, quando a condição se tornar falsa, ou seja, "a" for igual ou maior que "b", o processamento da rotina será interrompido e desviado para fora do loop. Por isso, não podemos esquecer de incrementar o valor de "a" para que o loop não se torne infinito.

```
while ( a < b ) do
  begin

     // Ao usar o while
     // não podemos esquecer de
     // incrementar a variável
     // de contagem
     a:=a+1;

     plot(a);

  end;
```

Laço de repetição FOR

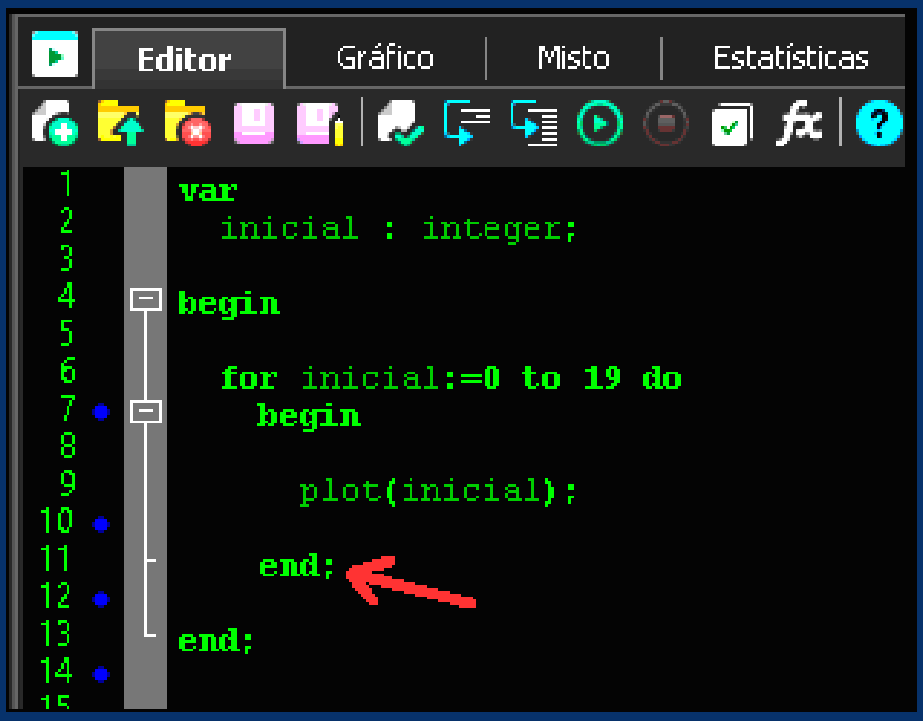
Usamos esse laço de repetição quando queremos repetir alguma parte do código uma quantidade específica de vezes. Sendo muito útil na programação de indicadores e estruturas de gerenciamento de risco. Um FOR loop precisa de duas informações para funcionar:

1. Valor inicial
2. Valor Final

```
for ValorInicial:=0 to ValorFinal do
  begin
     //instruções
  end;
```

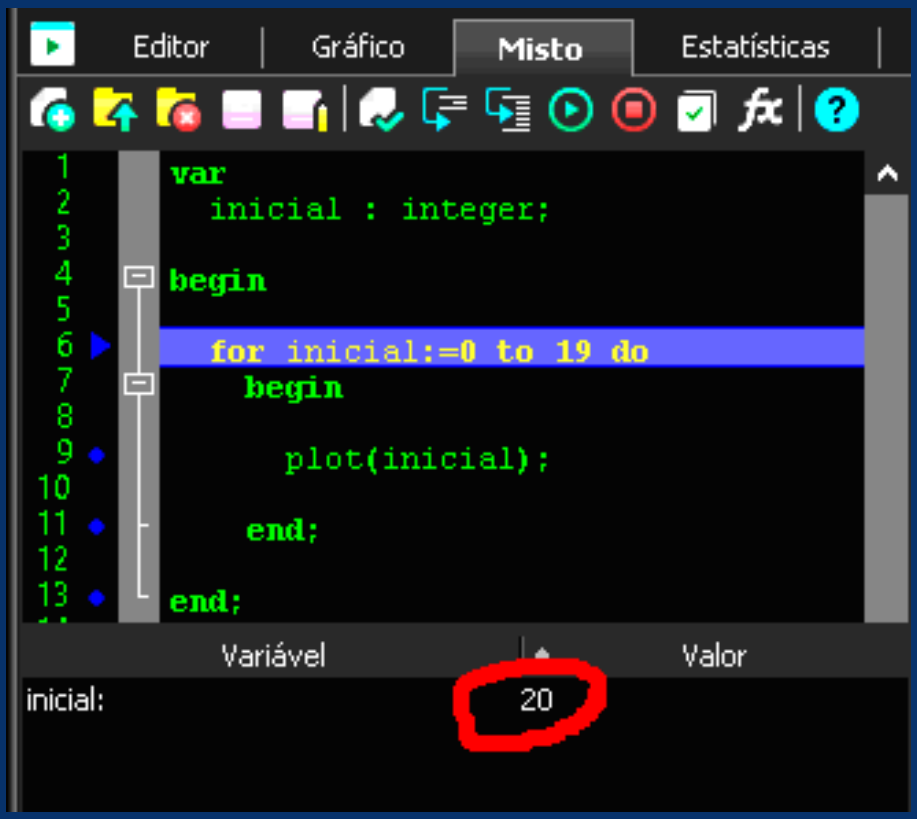
É importante ressaltar que o FOR, diferente do WHILE, realiza o incremento automaticamente, por isso não precisamos nos preocupar com essa parte quando usamos esse tipo de laço de repetição.

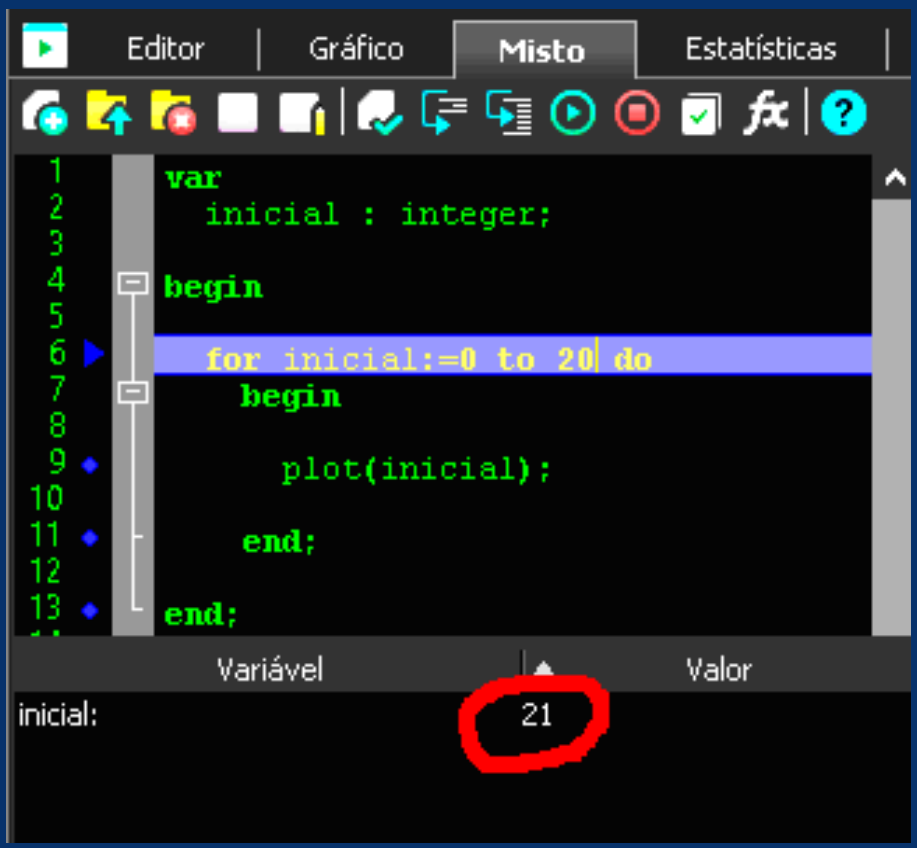
Outro ponto importante é que se quisermos realmente parar o laço em um número específico, precisaremos selecionar esse número, porém diminuir um. Por exemplo, digamos que queremos contar de 0 até 20, obtendo como valor final o número 20. Para fazer isso devemos selecionar como valor final o número 19, observe:



Sempre que o código atingir o end (ponto final do laço) marcado com a seta vermelha e voltar ao início do laço, ocorrerá um incremento na variável inicial. Quando inicial estiver valendo 19 ele ainda entrará do laço e atingirá o ponto final onde ocorre o incremento, elevando o valor de 19 para 20.

Por esse motivo, se colocássemos o laço de 0 até 20, o valor final seria 21, extrapolando o valor 20. Observe abaixo as imagens comparando essas duas situações:





Um exemplo muito comum do uso desse laço de repetição é o código de uma média móvel simples:

```
input
  periodoMedia(9);
  precoFecha(close);

var
  referencia : integer;
  soma : float;

begin

  if (CurrentBar>=periodoMedia-1) then
    begin

      soma:=0;

      for referencia:=0 to periodoMedia-1 do
        begin
          soma:=soma+precoFecha[referencia];
        end;

      plot(soma/periodoMedia);

    end;

end;
```

Utilizando as functions

Quando queremos executar uma tarefa específica repetidas vezes podemos criar uma função que realiza determinada tarefa e chamar ela sempre que necessária. Uma forma fácil de entender é pensar na média móvel. Sempre que escrevemos mediaExp(9,close) estamos chamando uma função já presente no Editor que calcula a média móvel exponencial.

No caso, criaremos nossas funções abaixo das variáveis da estratégia utilizando a palavra reservada function. Precisamos atribuir um nome a função, quantos parâmetros ela vai receber, qual o tipo de cada parâmetro e, por último, qual o tipo de dado que ela vai retornar. Observe:

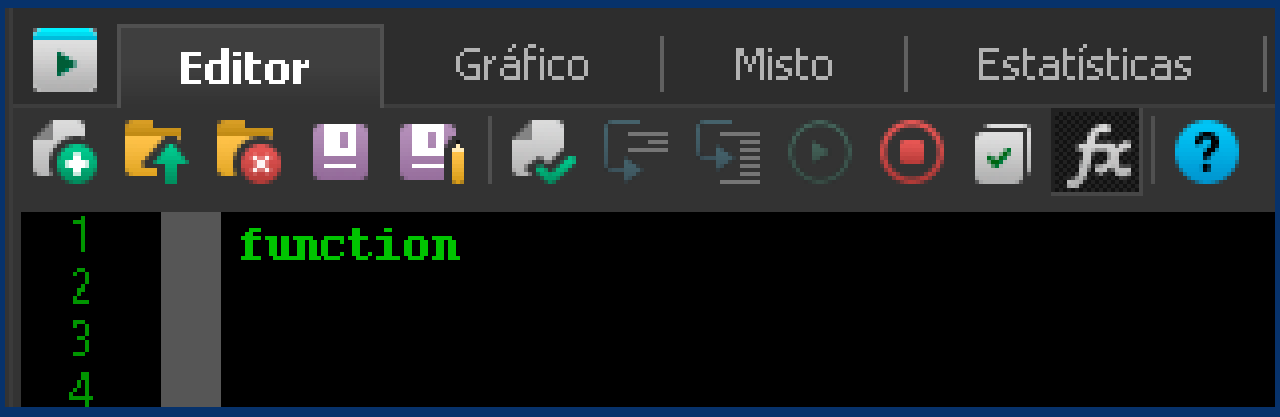
```
function somar(numero_1 : tipo; numero_2 : tipo; ) : tipo retornardo;
  begin

    //O que a função vai fazer fica aqui

  end;
```

Acima apenas podemos ver como é a sintaxe na criação de uma função mas para facilitar o aprendizado vamos criar uma função para somar dois números:

1. Escreva a palavra function (Essa é a parte mais difícil);



2. Atribua um nome a função (Começa a ficar mais fácil) ;



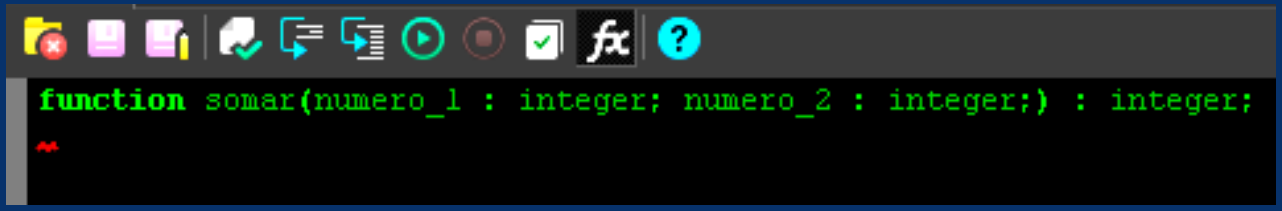
3. Abra os parênteses (Agora voltou a ficar mais difícil) ;



4. A pior parte já passou. Agora só precisamos imaginar quantos números queremos somar e quais os tipos desses números. Por exemplo, vamos somar um número A e um número B, e os dois serão números sem vírgula, ou seja, números inteiros;



5. Feito isso, vamos pensar: "Se eu somar um número inteiro com outro número inteiro, o que eu vou obter?" A resposta é clara: "Um número inteiro!!!". É impossível somar 1 + 1 e obter 1,65487112. Então, nossa função retorna um INTEGER;



6. Agora vamos abrir o corpo da função com um BEGIN e encerrar com o END sem esquecer do ponto e vírgula;

```
function somar(numero_1 : integer; numero_2 : integer;) : integer;
begin

end;
```

7. Atribuimos ao nome da função o cálculo para somar os dois números;

```
function somar(numero_1 : integer; numero_2 : integer;) : integer;
  begin
    somar:=numero_1 + numero_2;
  end;
```

8. A função está pronta para ser usada, vamos utilizar o comando plot no corpo do código para chamar a função e fazer ela somar dois números que vamos colocar dentro dos parênteses;

```
function somar(numero_1 : integer; numero_2 : integer;) : integer;
  begin
    somar:=numero_1 + numero_2;
  end;

begin

  plot(somar(3,4));

end;
```

9. Quando executamos o código, a função somar vai utilizar os dois números que colocamos dentro dos parênteses e somar, retornando o número inteiro 7. Observe abaixo o resultado:



Entendeu? Certamente, apesar de ser um conceito interessante, ele não afeta em nada a programação de estratégias no ProfitChart. É totalmente possível nem saber que as functions existem e programar diversos setups. Por isso, fique tranquilho se não conseguiu entender.

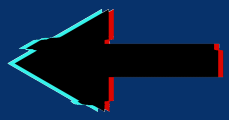
CÓDIGO LIVRE®

PROGRAMAÇÃO DE ESTRATÉGIAS

O primeiro canal oficial de programação avançada no ProfitChart!



Lembra que a poucas páginas atrás foi dito que ao usar a média móvel no código estamos usando uma função já pronta? Abaixo temos algumas das principais funções já prontas para usar nos códigos:

Funções para Back-testing 

BuyAtMarket: Realiza envio de ordem de compra à mercado;

BuyPrice: Retorna o valor da última operação de compra;

BuyStop(Stop, Limite): Realiza envio de ordem de compra stop;

BuyToCoverStop(Stop, Limite): Realiza o envio de ordem de compra stop para fechar posição;

BuyToCoverAtMarket: Realiza o fechamento de uma operação de venda;

IsBought: Retorna verdadeiro caso o cliente esteja posicionado na compra;

IsSold: Retorna verdadeiro caso o cliente esteja posicionado na venda;

SellPrice: Retorna o valor da última operação de venda;

SellToCoverAtMarket: Realiza uma ordem de venda à mercado caso exista uma posição de compra;

SellToCoverStop(Stop, Limite): Envia uma ordem stop caso exista uma posição de compra;

SellShortAtMarket: Envia ordem de venda à mercado para abrir posição;

SellShortStop(Stop, Limite): Abre uma posição de venda enviando uma ordem do tipo Stop.

Funções de Calendário e Tempo

CalcDate(Data referência, Data de alteração): Retorna o cálculo de adição ou subtração de datas

CalcTime(Hora referência, Tempo de alteração): Retorna o cálculo de adição ou subtração de horas;

CloseD(Número de dias anteriores): Retorna o valor de fechamento de determinado dia anterior;

CloseM(Número de meses anteriores): Retorna o valor de fechamento de determinado mês anterior;

CloseW(Número de semanas anteriores): Retorna o valor de fechamento de determinada semana anterior;

CloseY(Número de anos anteriores): Retorna o valor de fechamento de determinado ano anterior;

DaysToExpiration(Mês, Ano desejado - 1900): Retorna o número de dias que existem entre data especifica e dia atual;

HighD(Número de dias anteriores): Retorna o valor de máxima de determinado dia retroativo;

HighM(Número de dias anteriores): Retorna o valor de máxima de determinado mês retroativo;

HighW(Número de meses anteriores): Retorna o valor de máxima de determinada semana anterior;

HighY(Número de anos anteriores): Retorna o valor de máxima de um candle de determinado ano anterior;

LowD(Número de dias anteriores): Retorna o valor de mínima de determinado dia retroativo;

LowM(Número de meses anteriores): Retorna o valor de mínima de um candle mensal anterior determinado;

LowW(Número de semanas anteriores): Retorna o valor de mínima de um candle semanal anterior determinado);

LowY(Número de anos anteriores): Retorna o valor de mínima de um candle anual anterior determinado;

LastCalcDate: Retorna a data do último candle completo do gráfico;

LastCalcTime: Retorna o tempo de conclusão do último candle;

LastDayOfMonth(Mês de referência): Retorna o valor do último dia do mês de referência;

Next3rdFriday(Mes): Retorna quantos dias úteis faltam para a terceira sexta feira do determinado mês;

OpenD(Número de dias anteriores): Retorna o valor de abertura de determinado dia retroativo;

OpenM(Número de meses anteriores): Retorna o valor de abertura de determinado mês retroativo;

Funções Gráficas

Alert(Cor): Aciona um alarme quando
ocorrer situação e colore candle atual;

AvgPrice: Retorna a média entre Abertura,
Máxima, Mínima, Fechamento de determinado candle;

CurrentBar: Retorna o índice atual da
barra (candle);

C_Doji (Percentual): Retorna valor de 0 ou 1
caso o candle seja um Doji com o percentual determinado;

C_ShootingStar(Tamanho, Fator):
Retorna o valor de 0 ou 1 caso o candle seja um Shooting Star de acordo com o
Tamanho e o Fator;

Highest(Série, Período): Retorna o
maior valor da série dentro do período;

HighestBar(Série, Período): Retorna o
índice do maior valor da série no período;

LastBarOnChart: Retorna se é a última
barra do gráfico;

Leader: Retorna se ponto médio é maior que mínima ou maior que máxima de candle anterior;

Lowest(Série, Período): Retorna o menor valor da série dentro no período;

LowestBar(Série, Período): Retorna o índice da barra com o menor valor da série no período;

MedianPrice: Retorna a média entre a máxima e a mínima de determinado candle;

PaintBar(cor): Colore as barras do gráfico;

Plot: Desenha o indicador de acordo com o gráfico;

Range: Retorna o valor de Máxima menos Mínima do determinado candle;

RGB(RED, GREEN, BLUE): Colore de acordo com os valores de RGB;

TypicalPrice: Retorna o valor médio entre a máxima, mínima e fechamento de determinado candle.

Indicadores para as Estratégias

2MVPower: Retorna status do indicador
2MV Power;

AVGTrueRange(Período): Retorna o valor
do indicador True Range;

AccuDistr: Retorna o valor do indicador
Acumulação/Distribuição;

AccuDistrW: Retorna o valor do indicador
Acumulação/Distribuição Williams;

ADX(Período, Período da Média): Retorna o
valor do indicador ADX;

ArmsEaseOfMov(Média, Tipo de Média):
Retorna o valor do indicador Arms Ease of Movement;

AroonLin(Periodo)|Linha Desejada|: Retorna
o valor do indicador Aroon Linha, de acordo com o período e de acordo com a
linha desejada;

AroonnOsc(Período): Retorna o valor do
indicador Aroon Oscilador;

BalanceOfPower(Média, Tipo de Média):
Retorna o valor do indicador Balança de poder;

BearPower(Período,Média, Tipo de Média):
Retorna o valor do indicador Bear Power;

BollingerBands(Desvio, Média, Tipo de
Média)|Linha Desejada|: Retorna o valor da linha da Banda de Bollinger
de acordo com a linha desejada;

BollingerBandW(Desvio, Média, Tipo de
Média): Retorna o valor do indicador Bollinger Band Width;

BollingerBPerc(Desvio, Média, Tipo de
Média): Retorna o valor do indicador Boolinger b%;

BullPower(Período,Média, Tipo de Média):
Retorna o valor do indicador Bull Power;

CCI(Período): Retorna o valor do indicador
CCI;

ChaikinMoneyFlow(Período): Retorna
o valor do indicador Chaikin Money Flow;

ChaikinOsc(Média Longa, Média Curta):
Retorna o valor do indicador Oscilador Chaikin;

C_Doji(Percentual): Identifica se o candle é
um Candlestick "Doji;

DarvasBox: Retorna o valor do indicador
Darvas Box;

DiDiIndex(Média de Referência, Tipo de média de referência,Média1, Tipo de Média1, Media2, Tipo Média2)|Linha
Desejada|:Retorna o valor da linha selecionada do indicador DiDi
Index;

DiPDiM(Período)|Linha Desejada|: Retorna o
valor do indicador DI+/DI- de acordo com a linha desejada;

DonchianCH(Periodo)|Linha Desejada|:
Retorna o valor do indicador Canal Donchian de acordo com a linha desejada;

Envelope(Percentual, Média, Tipo de Média)|Linha
Desejada|: Retorna o valor do Retorna o valor da linha do indicador Envelope;

FastStochastic(Período): Retorna o valor do indicador Estocástico Rápido;

FinancialVol(VolumeProjetado): Retorna o Valor do volume financeiro com ou sem o Volume Projetado;

ForceIndex(Média, Tipo de média): Retorna
o valor do indicador Force Index;

FrassonATR(Fator, Período MaxMin, Período ATR)|Linha Desejada|: Retorna o valor do indicador Frasson ATR;

FrassonVH(Fator, Período MaxMin, Período VH)|Linha Desejada|: Retorna o valor do indicador Frasson VH;

FullStochastic(Período): Retorna o valor do indicador Estocástico Pleno;

FuraChao(Coeficiente, Deslocamento): Retorna o valor do indicador Fura Chão;

FuraTeto(Coeficiente, Deslocamento): Retorna o valor do indicador Fura Teto;

HeikinAshi(Média, Tipo de Média)|Dados desejados|: Retorna o valor do indicador HeikinAshi de acordo com a linha desejada;

HiLoActivator(Período)|Linha Desejada|: Retorna o valor do HiLo Activator de acordo com a linha desejada;

HistoVolatility(Média, Tipo da média): Retorna o valor do indicador Volatilidade Histórica;

IchimokuCloud(Tenkan-Sen, Kijun-Sen, Senkou Span B)|Linha Desejada|: Retorna o valor do indicador Ichimoku Cloud de acordo com a linha desejada;

ImpliedVolatility(Black&Scholes, Americana): Retorna o cálculo do indicador Volatilidade Implícita, dependendo do tipo de cálculo utilizado;

KeltnerCH(Desvio, Média, Tipo de Média)|Linha Desejada|: Retorna o valor do indicador Keltner Chanels, de acordo com a linha desejada;

Kvo(Média Longa, Média Curta, Sinal)|Dado Desejado|: Retorna o dado desejado do indicador KVO Linha & Histograma;

MACD(Média Longa, Média Curta, Sinal)|Dado Desejado|: Retorna o dado desejado do indicador KVO Linha & Histograma;

MFI: Retorna o valor do indicador Market Facilitation Index;

Momentum(Período, Média, Tipo da média): Retorna o valor do indicador Momentum;

MomentumStochastic(Período): Retorna o valor do indicador Momento Estocástico;

MoneyFlow: Retorna o valor do indicador Money Flow;

MoneyFlowIndex(Período): Retorna o valor do indicador Money Flow Index de acordo com o período utilizado;

NelogicaBottomFinder|Linha Desejada|: Retorna o valor do indicador Nelogica Bottom Finder de acordo com a linha Desejada;

NelogicaPullBackFinder|Linha Desejada|: Retorna o valor do indicador Nelogica PullBack Finder de acordo com a linha desejada;

OBV: Retorna o valor do indicador OBV;

OnBalanceTR: Retorna o valor do indicador On Balance True Range;

OpenInterest: o valor do indicador Contratos em aberto;

ParabolicSAR(Fator de Aceleração, Limite de aceleração): Retorna o valor do indicador SAR Parabolic;

Pivot(Normal,TrêsLinhas)|Linha Desejada|: Retorna o valor do indicador Pivot, de acordo com a linha selecionada;

PriceOsc(Média 1, Tipo de média 1, Média 2, Tipo de média 2): Retorna o valor do indicador Oscilador de preço, de acordo com as médias utilizadas;

PriorCote(Tipo): Retorna o valor do
indicador Prior Cote, de acordo com o tipo utilizado;

QuantityVol: Retorna o valor do indicador Volume Quantidade;
Rafi: Retorna o valor do indicador Rafi;

Ravi(Média Longa, Média Curta): Retorna o
valor do indicador Ravi de acordo com o período das médias desejadas;

Rsi(Período, Tipo):Retorna o valor do indicador
IFR;

RsiStochastic(Período): Retorna o
valor do indicador IFR Estocástico;

Roc(Período, Média, Tipo de Média): Retorna
o valor do indicador Roc;

SafeZoneDownTrend(Multiplicador, Média,
Deslocamento): Retorna o valor do indicador Stop

SlowStochastic(Período): Retorna o
valor do indicador Estocástico Lento;

StopATR(Desvio, Média, Tipo de Média)|Linha
Desejada|: Retorna o valor do indicador Stop ATR de acordo com a linha desejada;

Funções Matemáticas

ABS(Valor ou variável):Retorna o valor absoluto (sem sinal);

EXP(Valor ou variável): Retorna o valor de um Exponencial;

Factorial(Número): Retorna o fatorial do valor estabelecido (1*2*....n);

GCD(Valor1, Valor2) Retorna o maior denominador comum de dois números;

MinutesToTime(Minutos): Retorna o valor de minutos desde a meia noite em horas;

Power(Base, Expoente): Eleva valores nas determinadas potências;

Random(Limite): Retorna um valor aleatório de 0 até o limite;



Para ter acesso a lista completa com todos as funções inclusas na biblioteca do Editor acesse no seu Profit o caminho ajuda > conteúdo > sumário > Módulo de estratégias > Lista de Funções.

3° Aula – Entendendo o funcionamento do Editor

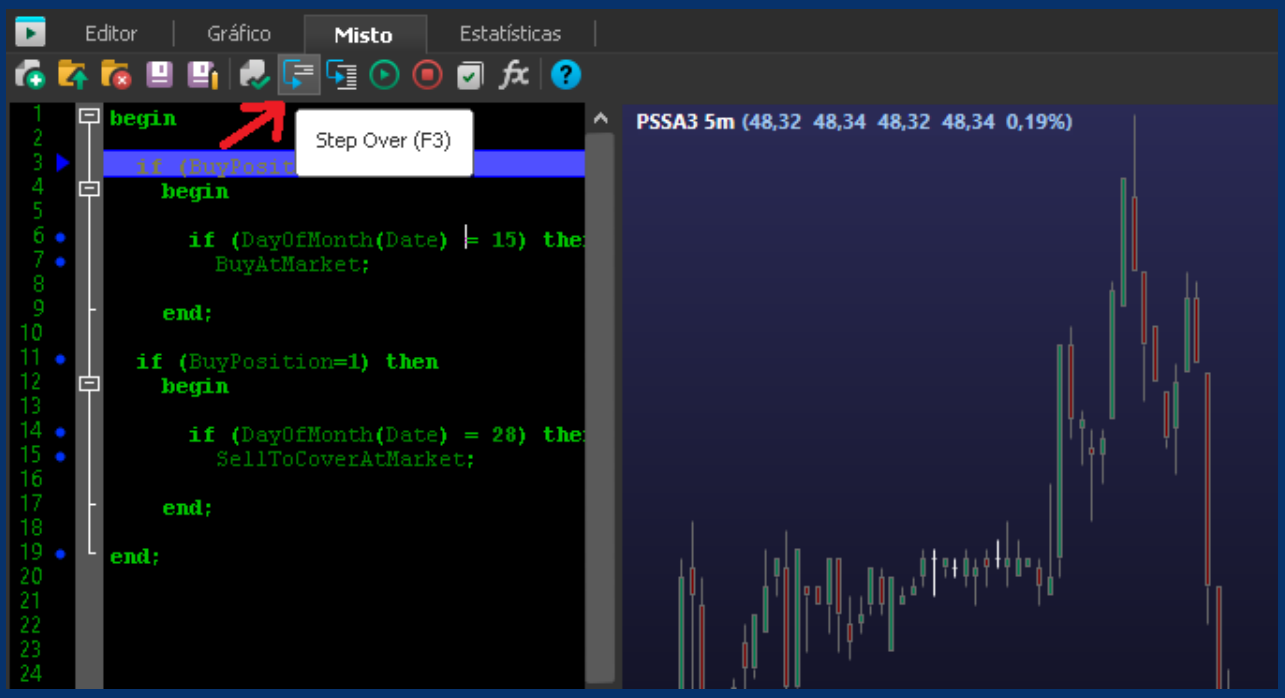
Na aula passada aprendemos sobre os laços de repetição e como as instruções dentro deles eram repetidas várias vezes enquanto a condição era verdadeira, e se eu dissesse que o Editor de estratégias funciona como um laço de repetição repetindo tudo o que você escreve todas as vezes que um novo candle surge?

Esse é o funcionamento do Editor, cada vez que uma barra surge ele lê o código e assim vai repetindo várias e várias vezes enquanto houver barras no gráfico. O nosso mercado é recente e por isso não temos muitos problemas com isso, escrevemos o código e executamos em todo o banco de dados rapidamente.

Porém, em outras plataformas de backtests como a que temos no famoso Trading View, é necessário iniciar a execução do código em ativos do mercado americano, a partir de um ano onde o compilador seja capaz de ler todos as barras do gráfico. Pois, como algumas ações americanas são muito antigas o loop acaba ficando enorme !

Conhecer esse funcionamento é muito importante quando queremos encontrar problemas no código, já que é possível executar a estratégia manualmente barra após barra e assim acompanhar a evolução da estratégia na íntegra. Fazendo isso podemos ver as variáveis mudando seus valores ao vivo e qualquer problema ficará exposto se tivermos paciência de acompanhar a execução.

Para iniciar a execução do código basta clicar repetidas vezes na tecla F3, fazendo isso o código começara a rodar e cada vez que surgir uma nova barra o compilador retorna ao topo do código.





Esse procedimento é muito útil para acompanhar a execução da estratégia e observar como o compilador está interpretando cada parte do código.

FIM DO MATERIAL COMPLEMENTAR

A partir de agora as aulas serão de conteúdo exclusivo dos tutoriais em vídeo presentes nos próximos módulos do curso "Desenvolvendo Setups para Vencer na Bolsa de Valores."

CÓDIGO LIVRE®
PROGRAMAÇÃO DE ESTRATÉGIAS
777

O PRIMEIRO CANAL DE PROGRAMAÇÃO AVANÇADA NO PROFITCHART APRESENTA

PROGRAMAÇÃO NO PROFITCHART 1.0

DOMINANDO O EDITOR DE ESTRATÉGIAS

Um curso detalhado desde o básico até o avançado sobre tudo que você precisa saber para construir estratégias e fazer diversos backtests utilizando a linguagem de programação da Nelogica !

ATENÇÃO

COPYRIGHT ARTHUR 777 AUTOR ©